CHAPTER 11

자연어처리 딥러닝 I

딥러닝 기초 · 단어 벡터를 이용한 추론 · 합성곱 신경망

학습 목표

- 딥러닝이 자연어처리를 어떻게 바꾸었는지 설명할 수 있다
- 신경망의 순전파·손실함수·역전파·최적화 과정을 이해한다
- 원핫 표현의 한계와 분산 표현(단어 임베딩)의 필요성을 설명할 수 있다
- 추론 기반 방법인 word2vec(CBOW·Skip-gram)의 구조와 학습 원리를 이해한다
- 합성곱 신경망의 기본 요소(커널·스트라이드·패딩·풀링)를 이해한다
- 1차원 합성곱을 이용한 문장 분류 모델을 직접 구현할 수 있다

11. 자연어처리 딥러닝 I

- 11.1 딥러닝은 자연어처리를 어떻게 바꾸었는가
- 11.2 신경망 기초
- 11.3 딥러닝 학습의 실제
- 11.4 단어를 벡터로 표현하기
- 11.5 분포 가설과 추론 기반 방법
- 11.6 word2vec (1) CBOW 11.7 word2vec (2) Skip-gram
- 11.8 word2vec 학습의 고속화
- 11.9 단어 벡터의 평가와 활용
- 11.10 합성곱 신경망의 기본 요소
- 11.11 텍스트를 위한 1차원 합성곱 신경망
- 11.12 전체 파이프라인 정리
- 11.13 확인 문제 11.14 요약 정리

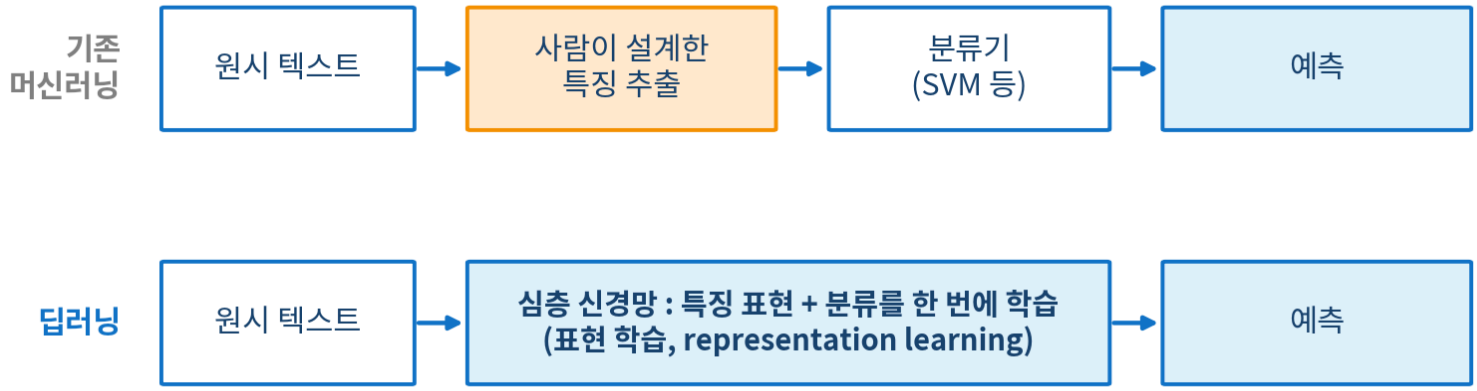
11.1 딥러닝은 NLP를 어떻게 바꾸었는가 (1/3)

- **자연어처리 방법론의 세 시대**
  - 규칙 기반 (1950~1980년대) : 언어학자가 문법 규칙과 사전을 직접 작성
    - 규칙의 수가 폭발적으로 늘어나고 예외 처리가 불가능
  - 통계 기반 (1990~2000년대) : 말뭉치에서 확률을 추정 (n-gram, HMM, CRF)
    - 사람이 설계한 특징(feature)에 성능이 좌우됨
  - 신경망 기반 (2010년대~) : 표현과 분류를 데이터로부터 함께 학습
    - 특징 공학의 부담이 사라지고, 대규모 말뭉치를 그대로 활용

11.1 딥러닝은 NLP를 어떻게 바꾸었는가 (2/3)

◦ **특징 공학(feature engineering)에서 표현 학습(representation learning)으로**

– 딥러닝은 원시 데이터로부터 과제에 유용한 표현을 스스로 찾아냄



기존 머신러닝 파이프라인과 딥러닝 파이프라인의 비교

11.1 딥러닝은 NLP를 어떻게 바꾸었는가 (3/3)

- **딥러닝이 가져온 네 가지 변화**
  - ① 이산 기호(단어)를 연속 벡터로 다루게 됨 → 단어 사이의 의미적 거리 계산 가능
  - ② 사람의 주석 없이도 학습 가능 → 대량의 말뭉치를 비지도 학습으로 활용
  - ③ 하나의 표현을 여러 과제에 재사용 → 사전 학습(pre-training)과 전이 학습
  - ④ 문제별 모델이 아니라 하나의 범용 구조로 수렴 → 오늘날의 대규모 언어모델
- **이번 주차의 위치**
  - ①과 ②를 담당하는 word2vec, 그리고 특징 추출기로서의 합성곱 신경망을 학습

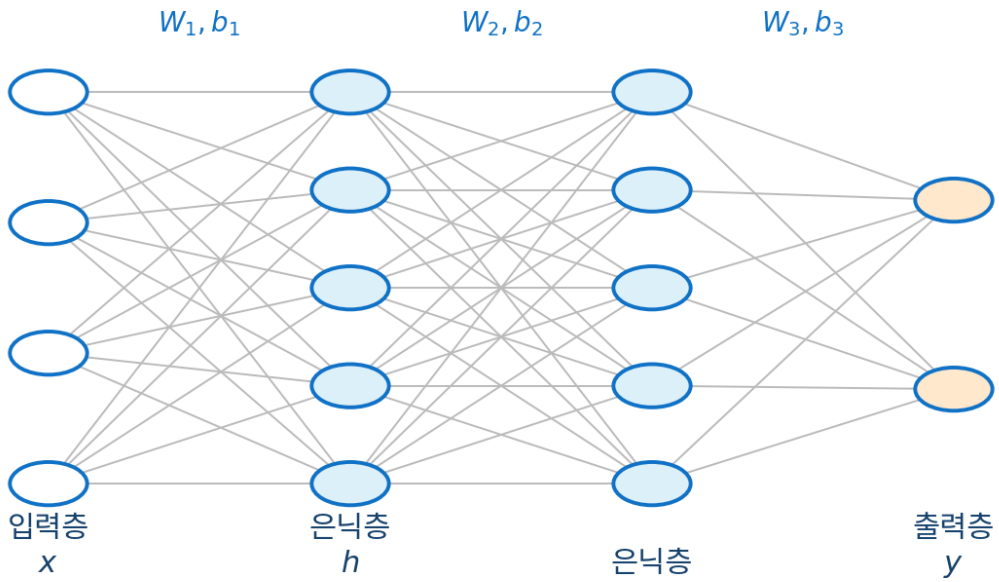
11.2 신경망 기초 (1/7)

- **퍼셉트론 : 신경망의 기본 계산 단위**
  - 퍼셉트론 : 입력의 가중합에 활성화 함수를 적용해 출력하는 계산 단위
    - $y = f(w_1x_1 + w_2x_2 + \ldots + w_nx_n + b)$
  - 단층 퍼셉트론은 선형 결정 경계만 만들 수 있어 XOR 문제를 풀 수 없음
- **다층 퍼셉트론(MLP)과 심층 신경망**
  - 은닉층을 추가하면 비선형 결정 경계를 표현할 수 있음
  - 은닉층이 2개 이상으로 깊어진 신경망을 심층 신경망(deep neural network)이라 함

11.2 신경망 기초 (2/7)

◦ **심층 신경망의 구조**

– 입력층 → 은닉층(여러 개) → 출력층으로 구성되며, 인접한 층은 가중치로 연결됨



심층 신경망의 기본 구조와 층별 연산

11.2 신경망 기초 (3/7)

- **왜 깊게 쌓는가**
  - 층이 깊어질수록 점점 더 추상적인 표현을 계층적으로 학습함
    - 텍스트의 예 : 문자 → 형태소/단어 → 구(phrase) → 문장의 의미
  - 같은 표현력을 얻는 데 필요한 뉴런 수가 얕은 신경망보다 훨씬 적음
    - 즉, 파라미터 효율이 높음 (다만 표현력이 커지는 만큼 과적합 위험도 커져 규제가 필요함 - 11.3)
- **대가**
  - 경사가 소실·폭주하기 쉽고, 학습에 더 많은 데이터와 연산이 필요함
  - 이를 완화하기 위해 ReLU, 정규화, 규제, Adam 등의 기법이 함께 발전

11.2 신경망 기초 (4/7)

- **순전파(forward propagation) 계산 예시**
  - 입력 x = [1.0, 2.0], 가중치와 바이어스가 아래와 같다고 가정

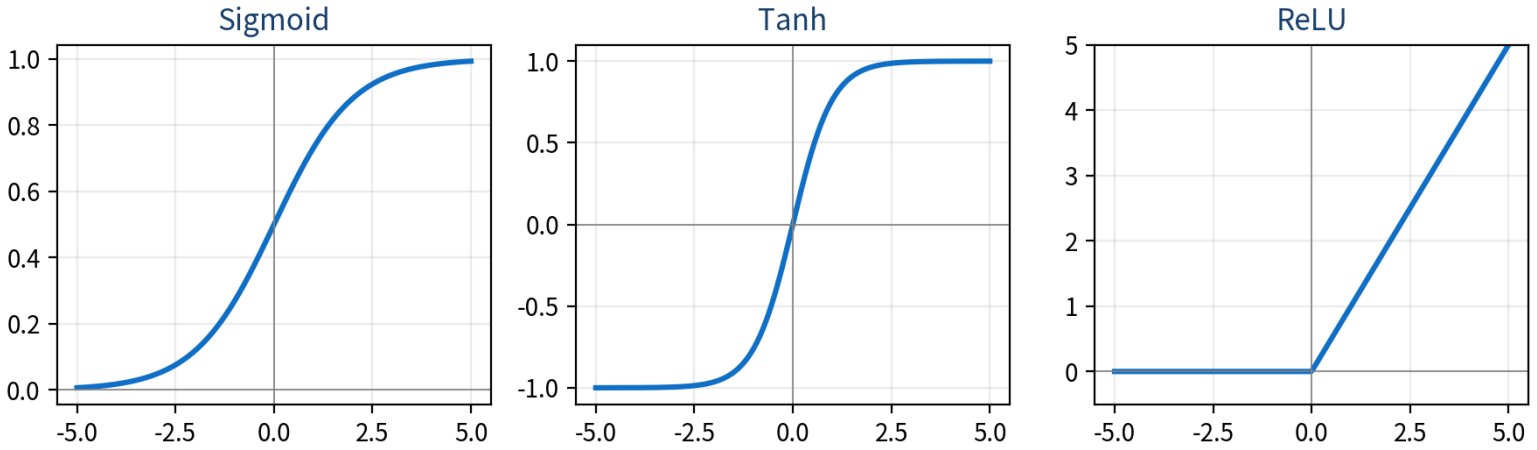
순전파는 입력에서 출력 방향으로 행렬곱과 활성화 함수를 번갈아 적용하는 과정이다

```
W1 = [[0.5, -0.3],      b1 = [0.1, 0.0]
      [0.2,  0.8]]
# 은닉층 : z1 = x·W1 + b1 = [0.5*1+0.2*2+0.1, -0.3*1+0.8*2+0.0]
#                         = [1.0, 1.3]
#          h  = ReLU(z1)  = [1.0, 1.3]
# 출력층 : z2 = h·W2 + b2  ->  y_hat = softmax(z2)
```

11.2 신경망 기초 (5/7)

- **활성화 함수(activation function)**
  - 활성화 함수가 없다면 층을 쌓아도 하나의 선형 변환과 같아짐

심층 신경망의 은닉층에는 경사 소실이 적고 계산이 빠른 ReLU를 기본으로 사용한다



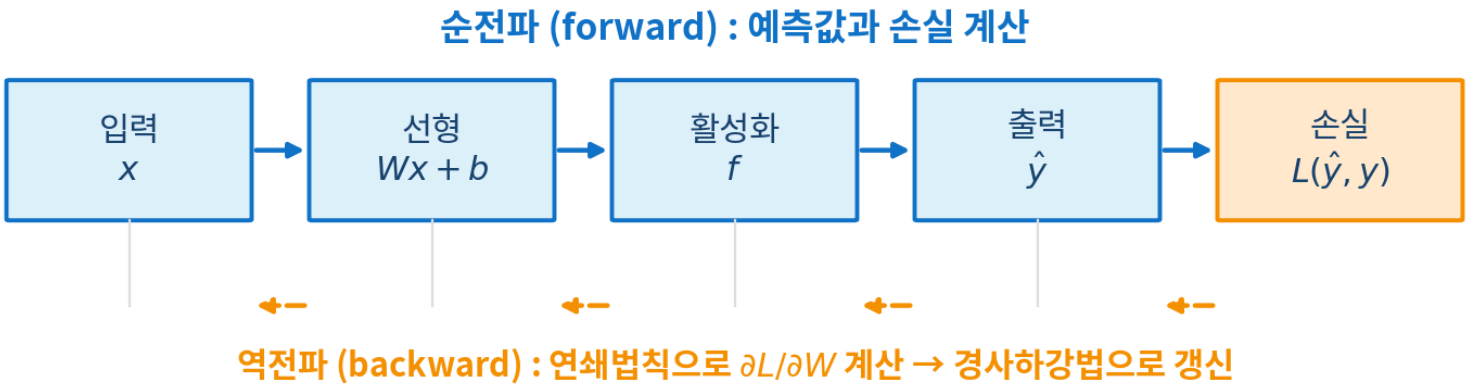
대표적인 활성화 함수

11.2 신경망 기초 (6/7)

- **손실 함수(loss function)**
  - 회귀 : 평균제곱오차(MSE) $L = (1/N)\Sigma(y - \hat{y})^2$
  - 이진 분류 : 이진 교차 엔트로피 $L = -[\, y \cdot \log \hat{y} + (1-y) \cdot \log(1-\hat{y}) \,]$
  - 다중 분류 : 소프트맥스 + 교차 엔트로피 $L = -\Sigma\, y_k \cdot \log \hat{y}_k$
- **소프트맥스 함수**
  - 출력 점수를 확률로 바꿈 : $\hat{y}_k = \exp(z_k) \,/\, \Sigma_j \exp(z_j)$
  - 단어를 예측하는 word2vec의 출력층도 어휘 크기 V의 소프트맥스를 사용함

11.2 신경망 기초 (7/7)

- **역전파와 경사하강법**
  - 연쇄법칙으로 각 가중치의 기울기를 출력층에서 입력층 방향으로 계산
  - 갱신식 : $W \leftarrow W - \eta \cdot \partial L/\partial W$ (η : 학습률)



11.3 딥러닝 학습의 실제 (1/4)

- **최적화 기법(optimizer)**
  - SGD : 미니배치의 기울기 방향으로 그대로 이동, 단순하지만 진동이 큼
  - Momentum : 이전 이동 방향을 관성처럼 누적해 진동을 줄임
  - AdaGrad · RMSProp : 파라미터별로 학습률을 자동 조절
  - Adam : Momentum과 RMSProp을 결합, 오늘날 자연어처리의 기본 선택
- **학습률(learning rate)**
  - 너무 크면 손실이 발산하고, 너무 작으면 학습이 지나치게 느려짐
  - 보통 1e-3에서 시작해 학습 곡선을 보며 조정 (스케줄링·워밍업도 사용)

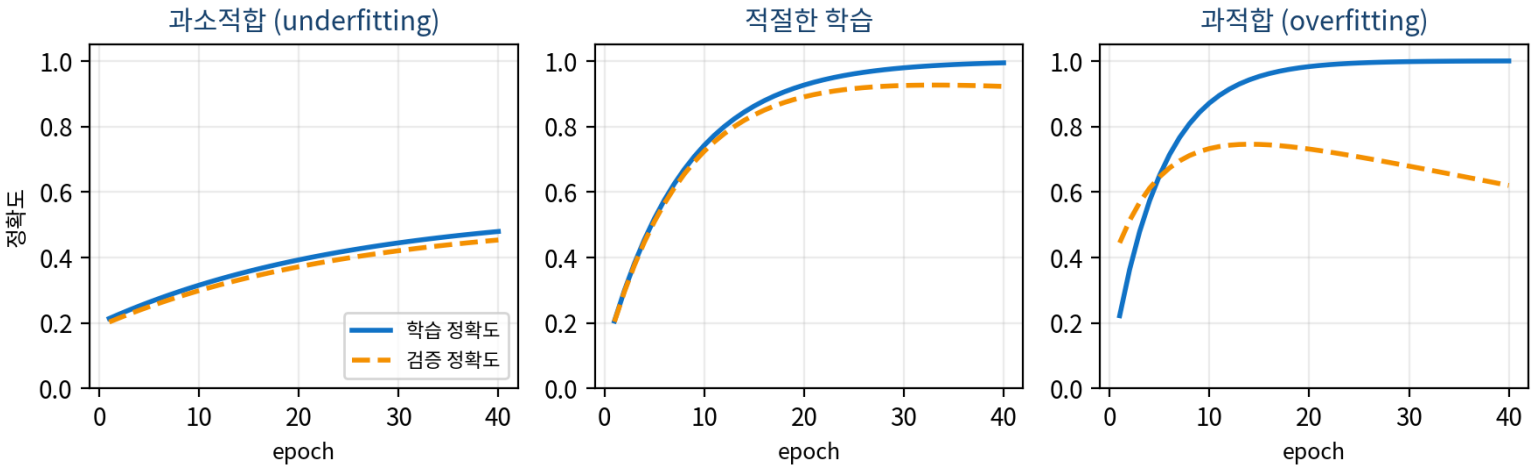
11.3 딥러닝 학습의 실제 (2/4)

- **미니배치 학습과 용어 정리**
  - 배치 크기(batch size) : 한 번의 갱신에 사용하는 데이터 수 (예 : 32, 64, 128)
  - 반복(iteration) : 가중치를 1회 갱신하는 것
  - 에폭(epoch) : 학습 데이터 전체를 1회 모두 사용한 것
    - 데이터 10,000개 · 배치 100 → 1에폭 = 100 iteration
- **데이터 세트의 분할**
  - 학습(train) / 검증(validation) / 평가(test) = 대략 60 / 20 / 20 %
  - 검증 데이터로 하이퍼파라미터를 고르고, 평가 데이터는 마지막에 한 번만 사용

11.3 딥러닝 학습의 실제 (3/4)

◦ 학습 상태의 판단

– 학습 곡선(정확도·손실)을 학습/검증에 대해 함께 그려 판단함



과소적합 · 적절한 학습 · 과적합의 학습 곡선 비교

11.3 딥러닝 학습의 실제 (4/4)

◦ **과적합을 막는 방법 (규제, regularization)**

- 데이터 증강 : 가장 근본적인 해결책, 텍스트에서는 동의어 치환·역번역 등
- 가중치 감쇠(L1 · L2) : 큰 가중치에 벌점을 주어 모델을 단순하게 유지
- 드롭아웃(dropout) : 학습 시 뉴런을 확률 p로 임의 제거해 앙상블 효과를 얻음
- 조기 종료(early stopping) : 검증 손실이 오르기 시작하면 학습을 멈춤
- 배치 정규화 : 층의 입력 분포를 안정화해 학습을 빠르고 안정적으로 만듦

```
model.add(Dropout(0.5))                          # 드롭아웃
model.add(Dense(64, kernel_regularizer=l2(1e-4)))  # L2 규제
```

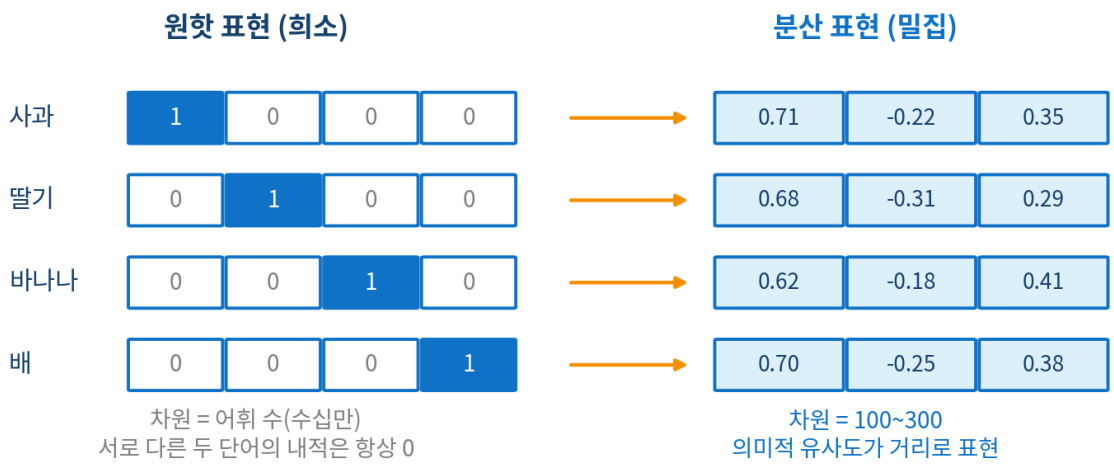
11.4 단어를 벡터로 표현하기 (1/3)

- **왜 벡터가 필요한가**
  - 신경망은 실수 벡터만 입력으로 받을 수 있음 → 문자열을 숫자로 바꾸어야 함
  - 첫 단계 : 학습 말뭉치의 단어를 모아 단어집(vocabulary)을 만들고 고유 번호를 부여
    - 등장 빈도 순으로 정렬하고, 희귀 단어는 <UNK> 하나로 묶는 것이 일반적
- **원핫 인코딩(one-hot encoding)**
  - 단어집 크기 V와 같은 길이의 벡터에서 자기 번호 위치만 1, 나머지는 0
    - ["사과","딸기","바나나","배"] → 사과 = [1,0,0,0], 딸기 = [0,1,0,0]

11.4 단어를 벡터로 표현하기 (2/3)

- **원핫 표현의 두 가지 한계**
  - ① 차원의 저주 : 어휘가 수십만 개면 벡터 차원도 수십만 → 계산·메모리 낭비
  - ② 의미를 담지 못함 : 서로 다른 원핫 벡터의 내적은 항상 0 (모든 단어가 등거리)

분산 표현(distributed representation) : 단어의 의미를 100~300차원의 밀집 벡터로 표현



11.4 단어를 벡터로 표현하기 (3/3)

- **분산 표현을 얻는 방법들**
  - 시소러스(thesaurus) 기반 : WordNet처럼 사람이 유의어 관계를 직접 정의
    - 문제 : 구축 비용이 크고, 시대 변화와 미묘한 어감 차이를 반영하지 못함
  - 통계 기반(counting) : 말뭉치의 동시발생 횟수를 세어 행렬을 만들고 차원 축소
  - 추론 기반(inference) : 신경망으로 "맥락 → 단어" 예측을 반복 학습 → word2vec
- **단어 임베딩의 목표**
  - 사람의 주석 없이 대량 말뭉치만으로, 여러 과제에 두루 쓰이는 벡터를 얻는 것

11.5 분포 가설과 추론 기반 방법 (1/3)

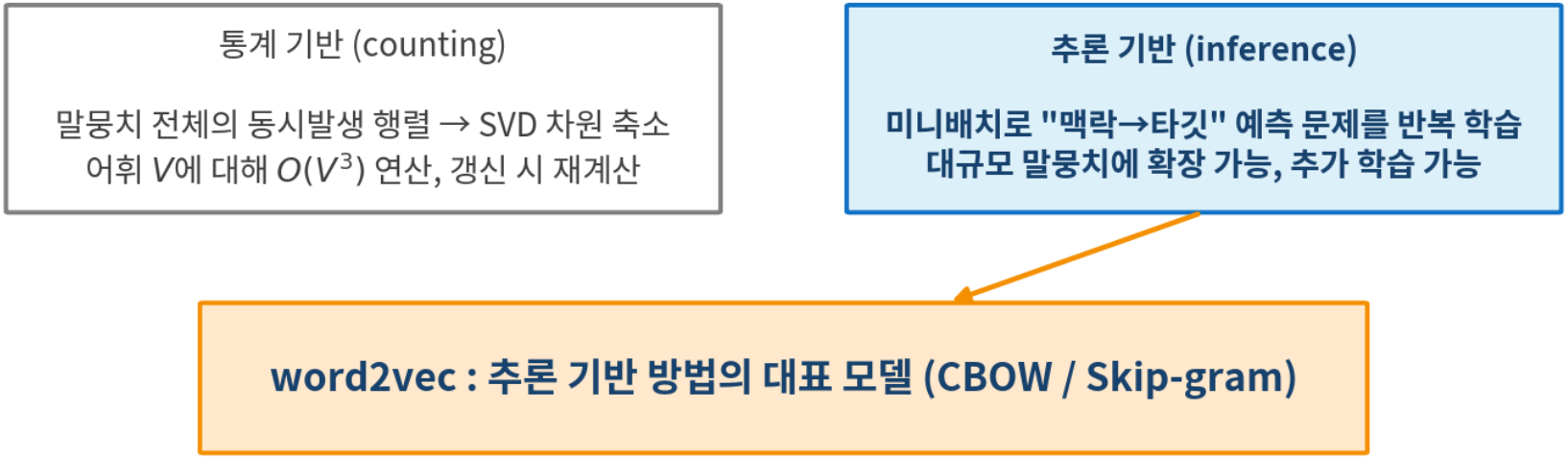
- **분포 가설 (distributional hypothesis)**
  - "단어의 의미는 그 주변 단어(맥락)에 의해 형성된다"
  - 예시 :
    - "나무에 열린 A가 맛있게 익었다" / "나무에서 딴 B를 맛있게 먹었다"
    - "C가 빠른 속도로 사라졌다"
  - A와 B는 서로 비슷한 의미(나무에 열리는 먹을 수 있는 것)일 것으로 예상
  - C는 A·B와 매우 다른 성질을 가질 것으로 예상
- **맥락(context)과 윈도우 크기**
  - 주목하는 단어 좌우로 몇 개까지 볼지를 윈도우 크기(window size)로 정함

11.5 분포 가설과 추론 기반 방법 (2/3)

- **통계 기반 방법의 개요와 한계**
  - 동시발생 행렬 : 각 단어의 맥락에 어떤 단어가 몇 번 등장했는지를 센 V×V 행렬
  - 점별 상호정보량(PPMI)으로 고빈도 단어의 영향을 보정
    - $\mathrm{PMI}(x,y) = \log_2 \{ P(x,y) / (P(x)\cdot P(y)) \}, \quad \mathrm{PPMI} = \max(0, \mathrm{PMI})$
  - 특이값 분해(SVD)로 밀집 벡터로 차원 축소
- **한계**
  - 어휘 V에 대해 SVD 계산량이 대략 $O(V^3)$ → 어휘 100만 개 규모에서는 비현실적 (절단 SVD를 쓰면 실용적으로 빠름)
  - 말뭉치가 추가되면 행렬을 다시 만들고 처음부터 다시 계산해야 함

11.5 분포 가설과 추론 기반 방법 (3/3)

- **추론 기반 방법 : 예측 문제로 바꾸어 풀기**
  - "you ? goodbye and I say hello" 의 ? 에 무엇이 들어갈지 맞히는 문제로 학습



통계 기반 방법과 추론 기반 방법의 비교

11.6 word2vec (1) CBOW (1/6)

- **word2vec (Mikolov et al., 2013)**
  - 추론 기반 단어 임베딩의 대표 모델로, 두 가지 구조를 제안함
    - CBOW (Continuous Bag-of-Words) : 맥락으로 중앙의 타깃 단어를 예측
    - Skip-gram : 타깃 단어로 주변 맥락 단어들을 예측
  - 계층이 두 개뿐인 얕은 신경망이지만, 계산량이 적어 대규모 말뭉치를 학습 가능
- **핵심 아이디어**
  - 예측을 잘 하도록 가중치를 학습시키면, 그 가중치가 곧 좋은 단어 벡터가 됨
  - 즉, 예측은 목적이 아니라 좋은 표현을 얻기 위한 수단(대리 과제)임

11.6 word2vec (1) CBOW (2/6)

- **학습 데이터 만들기 : 맥락과 타깃**
  - 말뭉치를 윈도우만큼 미끄러뜨리며 (맥락, 타깃) 쌍을 자동으로 생성

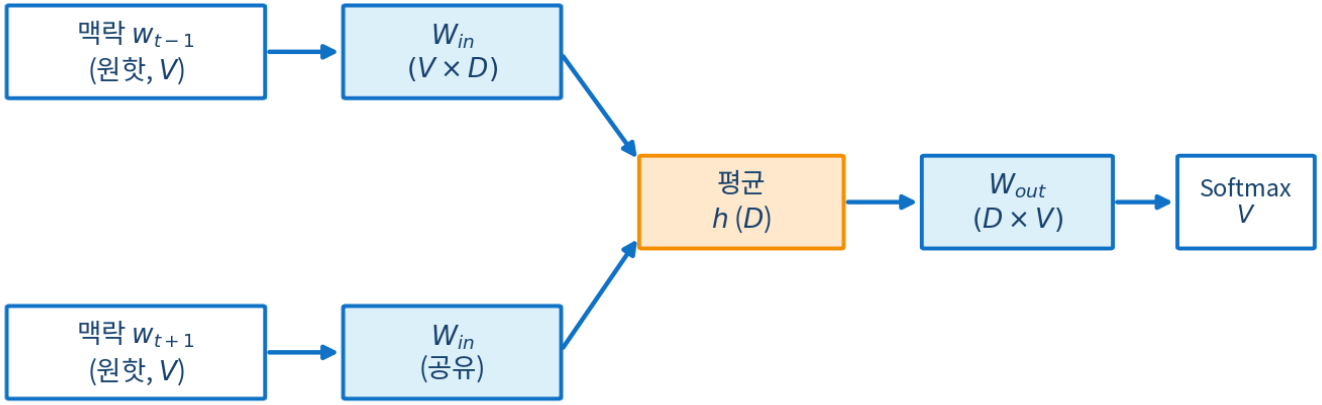


11.6 word2vec (1) CBOW (3/6)

◦ **CBOW의 신경망 구조**

- 입력 : 맥락 단어들의 원핫 벡터 (2×윈도우 개)
- 입력측 가중치 W_in(V×D)은 모든 맥락에서 공유되며, 은닉층은 그 평균

CBOW : 맥락으로 타깃을 예측



손실 : $L = -\log P(w_t \mid w_{t-1}, w_{t+1})$ **→ 학습이 끝난** W_{in}**의 각 행이 단어 벡터**

11.6 word2vec (1) CBOW (4/6)

- **순전파 수식**
  - 맥락 원핫 벡터 x_1, x_2 → 각각 $h_1 = x_1 W_{in}$, $h_2 = x_2 W_{in}$
  - 은닉층 $h = (h_1 + h_2) / 2$ (D차원, 보통 D = 100 ~ 300)
  - 점수 $s = h \cdot W_{out}$ (V차원) → 확률 $\hat{y} = \text{softmax}(s)$
  - 손실 $L = -\log \hat{y}[\text{타깃}]$ (교차 엔트로피)
- **파라미터 수**
  - $W_{in} : V \times D$, $W_{out} : D \times V$ → 어휘 10만, D=100이면 각각 1,000만 개

11.6 word2vec (1) CBOW (5/6)

- **학습 후 단어 벡터는 어디에 있는가**
  - W_in의 i번째 행 = i번째 단어의 분산 표현 → 일반적으로 이것을 단어 벡터로 사용
  - W_out의 i번째 열도 단어를 표현하지만, 관례적으로 W_in을 씀
    - 두 가중치를 더하거나 이어 붙여 사용하는 방법도 있음
- **CBOW라는 이름의 의미**
  - 맥락 단어들을 평균하므로 순서 정보가 사라짐 → "bag of words"
  - 대신 여러 맥락이 평균되어 잡음에 강하고 학습이 빠름

11.6 word2vec (1) CBOW (6/6)

- CBOW를 코드로 이해하기 (실습에서 직접 구현)

```
# 맥락(contexts)과 타깃(target) 만들기
def create_contexts_target(corpus, window_size=1):
    target = corpus[window_size:-window_size]
    contexts = []
    for idx in range(window_size, len(corpus) - window_size):
        cs = []
        for t in range(-window_size, window_size + 1):
            if t == 0: continue          # 자기 자신은 제외
            cs.append(corpus[idx + t])
        contexts.append(cs)
    return np.array(contexts), np.array(target)
```

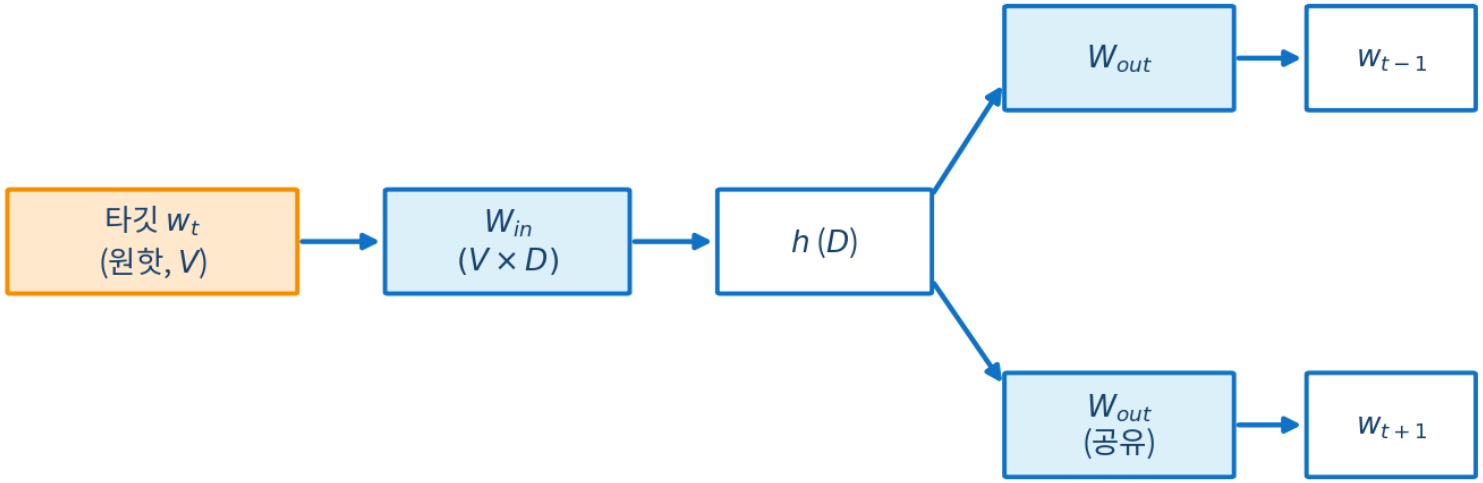
11.7 word2vec (2) Skip-gram (1/3)

- **Skip-gram : CBOW를 뒤집은 모델**
  - 중앙의 타깃 단어 하나를 입력해 주변 맥락 단어들을 각각 예측
  - 맥락이 2×윈도우 개라면 손실도 그 개수만큼 계산해 모두 더함
- **왜 더 어려운 문제인가**
  - 하나의 입력으로 여러 정답을 맞혀야 하므로 문제 자체가 더 까다로움
  - 더 어려운 문제를 풀게 하면 단어 벡터의 품질이 더 좋아지는 경향이 있음
    - 특히 희귀 단어의 표현이 CBOW보다 우수함

11.7 word2vec (2) Skip-gram (2/3)

◦ **Skip-gram의 신경망 구조**

Skip-gram : 타깃으로 맥락을 예측



손실 : $L = -\log P(w_{t-1} \mid w_t) - \log P(w_{t+1} \mid w_t)$ **→ 말뭉치가 클 때 CBOW보다 성능이 좋음**

11.7 word2vec (2) Skip-gram (3/3)

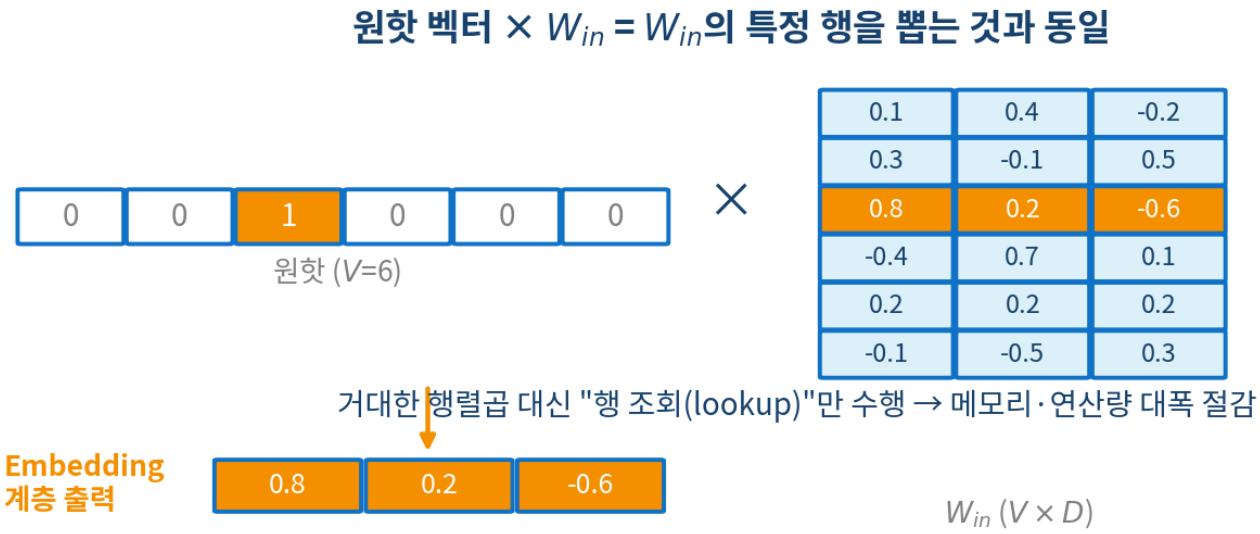
- **CBOW와 Skip-gram의 비교**
  - 학습 속도 : CBOW가 빠름 (한 쌍당 손실 1개)
  - 정확도 : 말뭉치가 클수록 Skip-gram이 유리, 특히 저빈도 단어와 유추 문제
  - 실무 관례 : 데이터가 작으면 CBOW, 크고 품질이 중요하면 Skip-gram
- **확률로 본 두 모델**
  - CBOW : $P(w_t \mid w_{t-1}, w_{t+1})$ 를 최대화
  - Skip-gram : $P(w_{t-1} \mid w_t) \cdot P(w_{t+1} \mid w_t)$ 를 최대화

11.8 word2vec 학습의 고속화 (1/5)

- **어휘가 커지면 어디가 병목인가**
  - ① 입력층의 원핫 벡터와 W_in의 행렬곱 (V차원 × V×D 행렬)
  - ② 은닉층과 W_out의 행렬곱 (D × D×V)
  - ③ 어휘 V개 전체에 대한 소프트맥스 계산
- **해결책**
  - ① → Embedding 계층 (행 조회로 대체)
  - ②③ → 네거티브 샘플링 (다중 분류를 이진 분류로 근사)
  - 추가로 고빈도 단어 서브샘플링으로 학습량 자체를 줄임

11.8 word2vec 학습의 고속화 (2/5)

- **Embedding 계층**
  - 원핫 벡터와의 행렬곱은 결국 특정 행 하나를 꺼내는 것과 같음



11.8 word2vec 학습의 고속화 (3/5)

- **네거티브 샘플링 (negative sampling)**
  - "어휘 V개 중 정답 고르기"를 "이 단어가 정답인가? (예/아니오)"로 바꿈

다중 분류 (Softmax)

어휘 전체 V개에 대해 점수와
확률을 모두 계산 → V가 커지면 병목

→

이진 분류 (Sigmoid)

"이 맥락에서 이 단어가 타깃인가?"
Yes/No 문제로 근사

네거티브 샘플링 : 정답 1개(positive) + 무작위로 뽑은 오답 k개(negative, 보통 5~20)에 대해서만 이진 분류 손실을 계산. 샘플링 확률 : $P(w_i) = \frac{f(w_i)^{0.75}}{\sum_j f(w_j)^{0.75}}$ (희귀 단어도 뽑히도록 0.75 제곱)

11.8 word2vec 학습의 고속화 (4/5)

- **네거티브 샘플링의 손실 함수**
  - 긍정 예 : $L^{+} = -\log \sigma(h \cdot v_target)$
  - 부정 예 : $L^{-} = -\Sigma_k \log \sigma(-h \cdot v_neg_k)$ $(k = 5 \sim 20)$
  - 최종 손실 $L = L^{+} + L^{-}$ → 계산량이 V에서 (k+1)로 감소
- **샘플링 확률의 0.75 제곱**
  - 단어 빈도를 그대로 쓰면 "은/는/이/가" 같은 조사만 뽑힘
  - 0.75 제곱을 하면 저빈도 단어가 뽑힐 확률이 상대적으로 높아짐

11.8 word2vec 학습의 고속화 (5/5)

- **주요 하이퍼파라미터**
  - 벡터 차원 D : 100 ~ 300 (크면 표현력↑, 과적합·비용↑)
  - 윈도우 크기 : 작으면(2~5) 문법적 유사성, 크면(10 이상) 주제적 유사성이 강해짐
  - min_count : 이 빈도 미만 단어는 학습에서 제외 (보통 5)
  - 네거티브 샘플 수 k : 작은 데이터 5~20, 큰 데이터 2~5
  - 에폭 수 : 보통 5 ~ 15

```
from gensim.models import Word2Vec
model = Word2Vec(sentences, vector_size=100, window=5,
                 min_count=5, sg=1, negative=5, epochs=10)  # sg=1 -> Skip-gram
```

11.9 단어 벡터의 평가와 활용 (1/4)

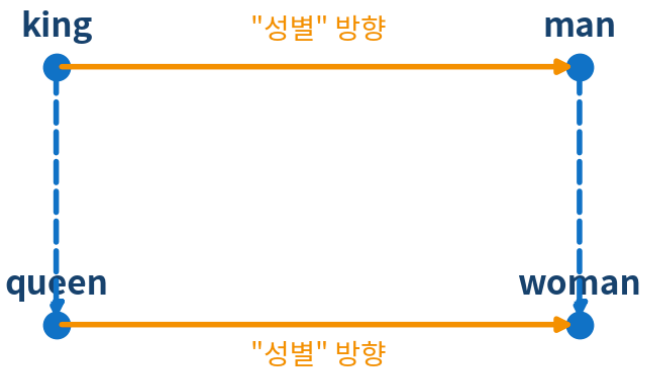
- **코사인 유사도 (cosine similarity)**
  - 두 벡터가 이루는 각도로 유사도를 측정 : $\cos(x, y) = (x \cdot y) / (\|x\| \|y\|)$
  - 값의 범위는 –1 ~ 1, 1에 가까울수록 의미가 유사함
  - 벡터의 길이(빈도의 영향)를 제거하므로 단어 유사도에 적합

```
def cos_similarity(x, y, eps=1e-8):
    nx = x / (np.sqrt(np.sum(x**2)) + eps)
    ny = y / (np.sqrt(np.sum(y**2)) + eps)
    return np.dot(nx, ny)
```

11.9 단어 벡터의 평가와 활용 (2/4)

- **유추 문제 (analogy task)**
  - "a : b = c : ?" 를 벡터의 덧셈·뺄셈으로 풀 수 있는지로 임베딩 품질을 평가

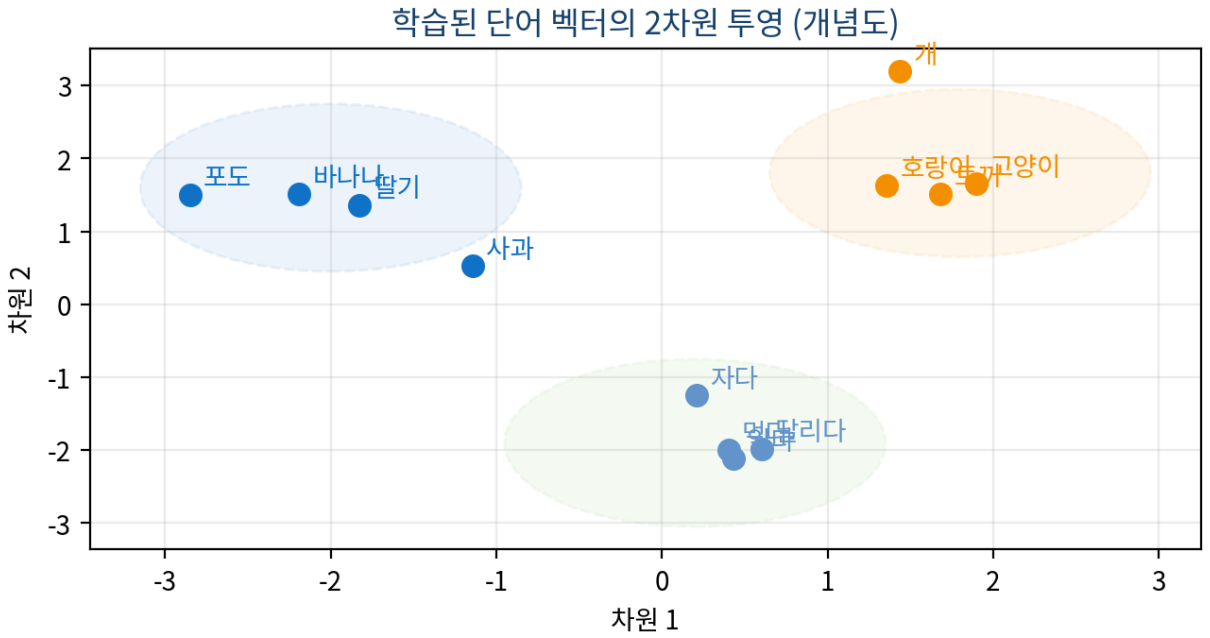
유추 문제를 푼다는 것은 "성별", "복수형", "국가–수도" 같은 관계가 벡터의 방향으로 인코딩되었음을 의미한다



vec("king") - vec("man") + vec("woman") ≈ vec("queen")

11.9 단어 벡터의 평가와 활용 (3/4)

- **단어 벡터 공간의 모습**
  - 의미가 비슷한 단어들이 공간상에서 군집을 이룸 (t-SNE·PCA로 2차원 투영)



학습된 단어 벡터의 2차원 투영 개념도

11.9 단어 벡터의 평가와 활용 (4/4)

- **한국어에서의 주의점**
  - 한국어는 교착어 → 어절에 조사·어미가 붙어 어휘 수가 폭발적으로 늘어남
    - "학교", "학교가", "학교에서", "학교로" 를 모두 다른 단어로 취급하게 됨
  - 따라서 전처리 단계에서 형태소 분석을 통해 어절을 형태소 단위로 분해함
    - 도구 : KoNLPy(Okt, Mecab), Kiwi 등
  - 또는 BPE·WordPiece 같은 서브워드 토큰화로 토큰 수와 성능의 균형을 잡음
- **활용**
  - 학습된 임베딩을 분류·번역·개체명 인식 모델의 입력층 초기값으로 재사용

11.9 보충 : word2vec 이후의 임베딩

- **GloVe (2014)**
  - 동시발생 행렬의 통계 정보를 직접 목적함수에 넣어 학습 (통계 + 추론의 결합)
- **FastText (2016)**
  - 단어를 문자 n-gram의 합으로 표현 → 학습에 없던 단어(OOV)도 벡터 생성 가능
  - 조사·어미 변화가 많은 한국어에 특히 유리함
- **정적 임베딩의 한계**
  - 한 단어에 벡터가 하나뿐 → "배(과일) / 배(선박) / 배(신체)"를 구분할 수 없음
  - 문맥에 따라 벡터가 달라지는 문맥 임베딩(ELMo, BERT)이 다음 단계 (13~14주차)

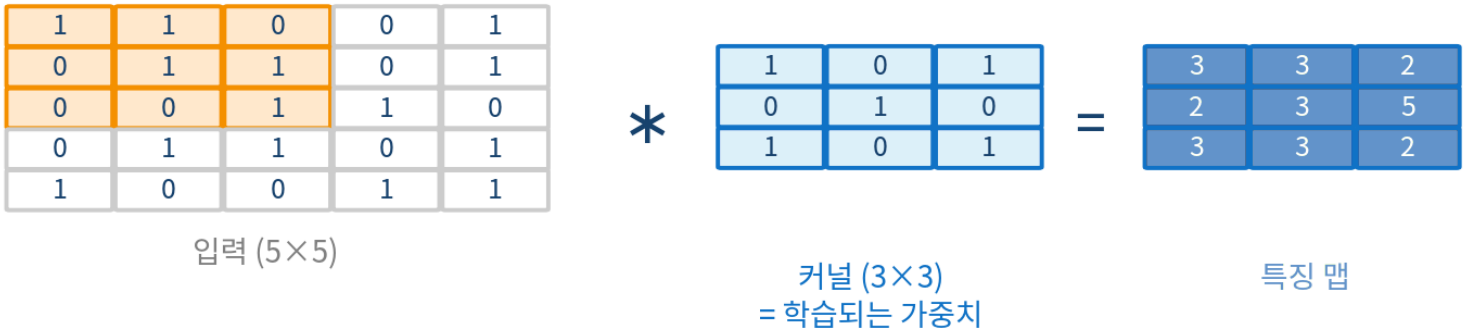
11.10 합성곱 신경망의 기본 요소 (1/8)

- **CNN이 등장한 배경**
  - 2차원 이미지를 1차원 벡터로 펼치면 이웃 픽셀의 공간 정보가 사라짐
  - 전결합층으로 처리하면 파라미터가 폭발 (28×28 → 은닉 1,000개면 78만 개)
- **CNN의 세 가지 특성**
  - 지역 수용 영역(local receptive field) : 인접한 입력만 함께 봄
  - 파라미터 공유(parameter sharing) : 같은 커널을 모든 위치에 재사용
  - 이동 등가성(equivariance) : 같은 패턴이면 위치가 달라도 같은 반응이 나옴
    - 풀링과 결합하면 위치 변화에 둔감해져 이동 불변성(invariance)을 얻음

11.10 합성곱 신경망의 기본 요소 (2/8)

◦ **합성곱(convolution) 연산**

– 커널(kernel)을 입력 위로 이동시키며 원소별 곱의 합을 계산



수용 영역(주황)과 커널을 원소별로 곱해 모두 더한 값 = 출력 한 칸 → 커널을 이동시키며 반복

11.10 합성곱 신경망의 기본 요소 (3/8)

- **커널(= 필터)의 의미**
  - 커널의 값이 바로 학습되는 가중치 → 무작위로 초기화되고 역전파로 갱신됨
  - 커널이 위치한 입력의 범위를 수용 영역(receptive field)이라 함
- **커널 크기와 필터 수**
  - 커널 크기 : 작으면(3×3) 세밀한 패턴, 크면(7×7) 넓은 범위의 패턴을 봄
  - 필터 수 : 커널을 몇 개 쓸지 → 출력 특징 맵의 개수(= 다음 층의 깊이)를 결정
  - 첫 층은 모서리·직선, 다음 층은 원·사각형, 더 깊은 층은 복잡한 형태를 학습

11.10 합성곱 신경망의 기본 요소 (4/8)

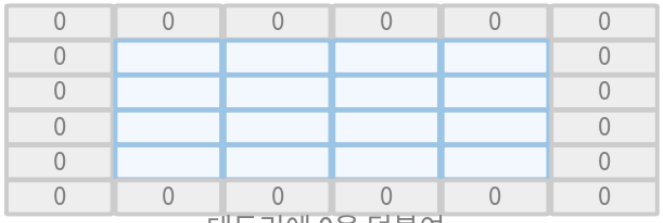
- **스트라이드(stride)와 패딩(padding)**
  - 스트라이드 : 커널이 한 번에 이동하는 칸 수
  - 제로 패딩 : 테두리에 0을 덧붙여 출력 크기를 유지하고 경계 정보를 보존

스트라이드 = 1



필터가 한 번에 이동하는 칸 수
값이 커지면 출력 크기가 줄어듦

제로 패딩 = 1



테두리에 0을 덧붙여
출력 크기를 입력과 동일하게 유지

11.10 합성곱 신경망의 기본 요소 (5/8)

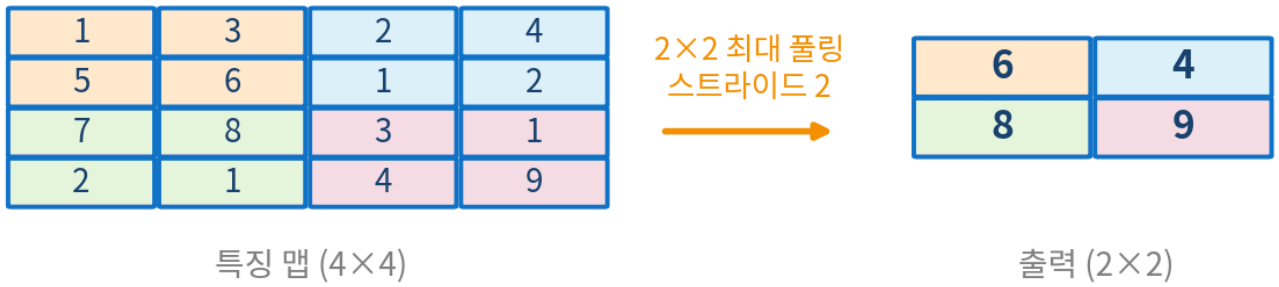
◦ **출력 크기 계산 공식**

- 출력 = ⌊ (입력 − 커널 + 2×패딩) / 스트라이드 ⌋ + 1
- 예 : 입력 28, 커널 3, 패딩 0, 스트라이드 1 → 출력 26
- 예 : 입력 28, 커널 3, 패딩 1, 스트라이드 1 → 출력 28 (크기 유지)

```
# 케라스에서의 합성곱 층
Input(shape=(28, 28, 1))                          # 입력 모양은 Input 층에 지정
Conv2D(filters=32, kernel_size=(3, 3), strides=1,
       padding='same', activation='relu')
#   padding='same' -> 스트라이드 1일 때 출력 = 입력 (일반적으로 ceil(입력/스트라이드))
#   padding='valid' -> 패딩 없음 (크기가 줄어듦)
```

11.10 합성곱 신경망의 기본 요소 (6/8)

- **풀링 층 (pooling layer)**
  - 통계 함수(최대·평균)로 특징 맵을 축소해 다음 층의 파라미터 수를 줄임



각 영역의 최대값만 남김 → 파라미터·연산량 감소, 작은 위치 변화에 강건해짐

11.10 합성곱 신경망의 기본 요소 (7/8)

- **풀링을 사용하는 이유**
  - 계산 복잡도와 파라미터 수를 줄여 과적합 위험을 낮춤
  - 작은 위치 변화에 둔감해져(이동 불변성) 일반화 성능이 좋아짐
  - 풀링 층에는 학습되는 파라미터가 없음
- **입력과 출력의 크기 변화 정리**
  - 합성곱 층 : 가로·세로는 (패딩에 따라) 유지 또는 감소, 깊이는 필터 수로 변경
  - 풀링 층 : 가로·세로는 감소, 깊이는 그대로 유지
  - 전결합층에 넣기 전에 Flatten으로 1차원 벡터로 펼침

11.10 합성곱 신경망의 기본 요소 (8/8)

- **CNN 전체 구조 요약**
  - 입력 → [합성곱 + 활성화 + 풀링] 반복 → Flatten → 전결합층 → 출력
  - 앞부분은 특징 추출기, 뒷부분은 분류기 역할을 담당

```
model = Sequential([
    Input(shape=(28, 28, 1)),
    Conv2D(32, 3, activation='relu'),
    MaxPooling2D(2),
    Conv2D(64, 3, activation='relu'),
    MaxPooling2D(2),
    Flatten(),
    Dropout(0.5),
    Dense(10, activation='softmax'),
])
```

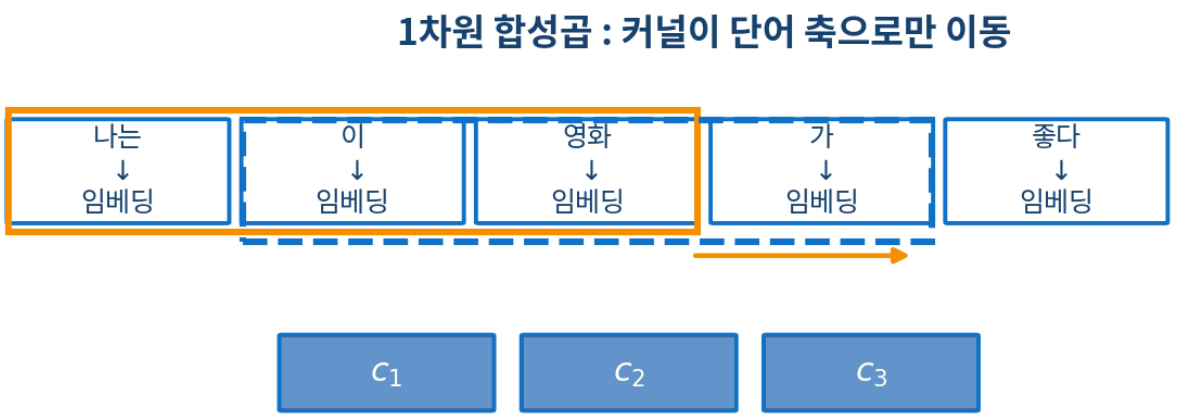
11.11 텍스트를 위한 1차원 CNN (1/6)

- **문장에 CNN을 쓸 수 있을까**
  - 문장을 임베딩하면 (문장 길이 × 임베딩 차원)의 2차원 행렬이 됨
  - 단, 임베딩 차원 축은 "공간"이 아니므로 그 방향으로 커널을 움직이면 의미가 없음
  - → 단어(시간) 축으로만 이동하는 1차원 합성곱을 사용함
- **왜 효과적인가**
  - 폭이 n인 커널은 n-gram 패턴 탐지기 역할을 함
    - "별로 재미 없다", "정말 최고" 같은 표현을 위치와 무관하게 잡아냄
  - 순환 신경망과 달리 병렬 계산이 가능해 학습이 매우 빠름

11.11 텍스트를 위한 1차원 CNN (2/6)

◦ **1차원 합성곱 연산**

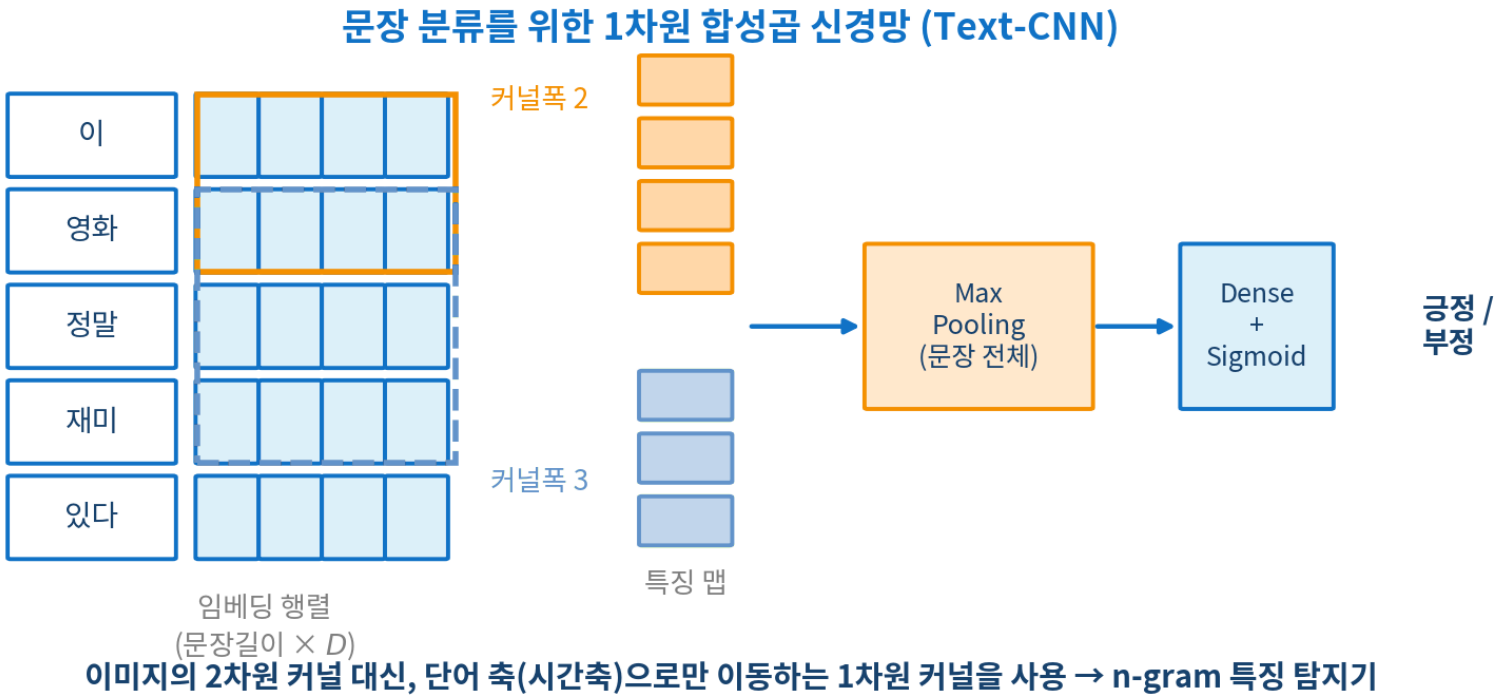
– 커널의 폭 = 함께 볼 단어 수, 커널의 높이 = 임베딩 차원 D (고정)



출력 길이 = 문장길이 - 커널폭 + 1 (padding="same" 이면 문장길이 유지)

11.11 텍스트를 위한 1차원 CNN (3/6)

- **Text-CNN (Kim, 2014) 구조**
  - 여러 크기의 커널(원 논문은 3·4·5-gram, 각 100개)을 병렬로 적용해 결과를 이어 붙임



11.11 텍스트를 위한 1차원 CNN (4/6)

- **전역 최대 풀링 (Global Max Pooling)**
  - 각 특징 맵에서 가장 큰 값 하나만 남김 → 문장 길이와 무관한 고정 길이 벡터
  - "해당 n-gram 패턴이 문장 어딘가에 강하게 존재했는가"를 하나의 값으로 요약
- **임베딩 층의 세 가지 선택**
  - 무작위 초기화 후 함께 학습 (데이터가 충분할 때)
  - 사전 학습된 word2vec을 넣고 고정 (trainable=False, 데이터가 적을 때)
  - 사전 학습된 값으로 초기화한 뒤 미세 조정(fine-tuning) → 보통 가장 좋음

11.11 텍스트를 위한 1차원 CNN (5/6)

◦ 케라스 구현 (감성 분석)

Text-CNN의 핵심은 서로 다른 폭(3·4·5)의 커널을 병렬로 적용해 결과를 이어 붙이는 것이다 (Functional API 필요)

```
inp = Input(shape=(max_len,), dtype="int32")
e   = Embedding(vocab_size, 100)(inp)          # 단어 임베딩 (V x 100)
br  = [GlobalMaxPooling1D()(Conv1D(100, k, activation="relu")(e))
       for k in (3, 4, 5)]                     # 폭 3·4·5 커널 각 100개
x   = Concatenate()(br)                        # 세 분기를 이어 붙임
x   = Dropout(0.5)(x)
x   = Dense(64, activation="relu")(x)
out = Dense(1, activation="sigmoid")(x)        # 긍정/부정 이진 분류
model = Model(inp, out)
model.compile(optimizer='adam', loss='binary_crossentropy',
              metrics=['accuracy'])
```

11.11 텍스트를 위한 1차원 CNN (6/6)

- **하이퍼파라미터 튜닝의 우선순위 (경험적)**
  - ① 커널 크기 조합 (2~5 중 선택) → ② 필터 수 (100~600) → ③ 드롭아웃 비율(0.3~0.5)
  - 활성화 함수는 ReLU, 풀링은 전역 최대 풀링이 대체로 안정적
- **다른 모델과의 비교**
  - CNN : 국소적 패턴에 강함, 매우 빠름, 긴 의존 관계는 잡기 어려움
  - RNN·LSTM : 순서와 장거리 의존 관계에 강함, 순차 계산으로 느림 (12주차)
  - Transformer : 어텐션으로 전 구간을 동시에 참조 (14주차)

11.12 전체 파이프라인 정리

- 이번 주차에서 배운 요소들이 어떻게 이어지는가

토큰화 → 사전 구축 → 임베딩 → 합성곱·풀링 → 분류 : 이후 배우는 모델도 이 골격을 공유한다



11주차 실습은 이 파이프라인의 3~6단계를 직접 구현한다

11.13 확인 문제

◦ 개념 점검

- ① 원핫 표현의 두 가지 한계는 무엇인가?
- ② 분포 가설을 한 문장으로 설명하고, 통계 기반과 추론 기반의 차이를 쓰시오.
- ③ CBOW와 Skip-gram의 입력·출력과 손실 함수를 각각 비교하시오.
- ④ 네거티브 샘플링이 어떤 병목을 어떻게 해결하는지 설명하시오.
- ⑤ 입력 길이 100, 커널 5, 패딩 0, 스트라이드 1인 Conv1D의 출력 길이는?
- ⑥ 전역 최대 풀링을 사용하면 어떤 이점이 있는가?

11.14 요약 정리

- **딥러닝 기초**
  - 순전파 → 손실 계산 → 역전파 → 경사하강법 갱신의 반복이 학습의 본질
  - ReLU·Adam·드롭아웃·조기 종료는 깊은 신경망을 학습시키기 위한 표준 도구
- **단어 벡터를 이용한 추론**
  - 분포 가설에 근거해 "맥락 ↔ 타깃" 예측을 학습하면 좋은 단어 벡터가 얻어짐
  - CBOW·Skip-gram, Embedding 계층과 네거티브 샘플링으로 대규모 학습이 가능
- **합성곱 신경망**
  - 커널·스트라이드·패딩·풀링이 기본 요소이며, 지역 패턴 추출기로 동작
  - 텍스트에서는 1차원 합성곱이 n-gram 탐지기 역할을 하여 문장 분류에 효과적

참고 자료

- **교재 및 도서**
  - 사이토 고키, 『밑바닥부터 시작하는 딥러닝 2』, 한빛미디어
  - 조준우, 『머신러닝·딥러닝 문제해결 전략』 / 텐서플로 공식 자연어처리 튜토리얼
- **논문**
  - Mikolov et al. (2013), Efficient Estimation of Word Representations in Vector Space
  - Mikolov et al. (2013), Distributed Representations of Words and Phrases
  - Kim (2014), Convolutional Neural Networks for Sentence Classification
- **실습 참고**
  - https://www.tensorflow.org/text/tutorials/word2vec
  - https://keras.io/examples/nlp/